

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: شایان عباسی

شماره دانشجویی: 04121040768022

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

پروژه 6: شبیه‌ساز گیت مترو

امکانات

- کارت مترو
- ورود و خروج مسافر
- سنسورهای گیت
- موتور باز و بسته شدن

کلاس‌ها

Passenger

Card

Sensor

Motor

Gate

GUI

نمایش باز و بسته شدن گیت

1. کتابخانه ها

```
import tkinter as tk
```

در این خط، کتابخانه `tkinter` را وارد کردم و نام مستعار `tk` به آن دادم. از این کتابخانه برای ساخت پنجره اصلی برنامه و تمام عناصر گرافیکی مانند `Canvas`، دکمه‌ها، `Label` و `Entry` استفاده کردم.

```
from tkinter import messagebox
```

از این خط برای وارد کردن ماژول `messagebox` از `tkinter` استفاده کردم. این ماژول پنجره‌های هشدار و پیام را فراهم می‌کند. وقتی موجودی کارت کافی نباشد از `showwarning` برای نمایش پیام خطا استفاده می‌شود.

۲. کلاس Card (مدل کارت مترو)

```
class Card:
```

```
    def __init__(self, card_id, balance):  
        self.card_id = card_id  
        self.balance = balance
```

این کلاس را برای مدل‌سازی کارت مترو طراحی کردم. هر کارت یک شناسه منحصر به فرد `card_id` و موجودی `balance` دارد. موجودی اولیه در هنگام ساخت کارت تنظیم می‌شود.

متد `has_balance` بررسی موجودی

```
    def has_balance(self):  
        if self.balance >= 2.0:  
            return True  
        else:  
            return False
```

این متد بررسی می‌کند آیا موجودی کارت برای یک سفر کافی است یا خیر. کرایه هر سفر ۲ دلار است. اگر موجودی کمتر از ۲ باشد `False` و در غیر این صورت `True` برمی‌گرداند. `Gate` قبل از باز کردن این متد را بررسی می‌کند.

متد deduct کسر کرایه

```
def deduct(self):  
    self.balance = self.balance - 2.0
```

این متد مبلغ کرایه را از موجودی کارت کسر می‌کند. هر بار که مسافر با موفقیت وارد می‌شود این متد فراخوانی شده و ۲ دلار از موجودی کم می‌شود.

۳. کلاس Passenger (مدل مسافر)

```
class Passenger:  
    def __init__(self, name, card):  
        self.name = name  
        self.card = card
```

این کلاس را برای مدل‌سازی مسافر طراحی کردم. هر مسافر یک نام و یک کارت مترو دارد. در متد `get_passenger` از روی اطلاعات فرم یک شیء `Passenger` ساخته می‌شود و به `Gate` داده می‌شود.

۴. کلاس Sensor (سنسور گیت)

```
class Sensor:  
    def __init__(self):  
        self.card_detected = False
```

این کلاس را برای شبیه‌سازی سنسور تشخیص کارت ساختم. `card_detected` وضعیت تشخیص کارت را نگه می‌دارد. در ابتدا `False` است یعنی هیچ کارتی شناسایی نشده.

مدهای detect / reset تشخیص و ریست

```
def detect(self):  
    self.card_detected = True  
  
def reset(self):  
    self.card_detected = False
```

متد **detect** وقتی کارت به سنسور نزدیک می‌شود فراخوانی می‌شود و وضعیت را **True** می‌کند. متد **reset** بعد از بسته شدن گیت وضعیت را به **False** برمی‌گرداند تا سنسور آماده تشخیص بعدی باشد.

۵. کلاس **Motor** موتور گیت

```
class Motor:
    def __init__(self):
        self.is_open = False
```

این کلاس را برای شبیه‌سازی موتور باز و بسته شدن گیت ساختم. **is_open** وضعیت فیزیکی گیت را نگه می‌دارد. در ابتدا **False** است یعنی گیت بسته است.

متدهای **open / close** باز و بستن گیت

```
def open(self):
    self.is_open = True
def close(self):
    self.is_open = False
```

متد **open** گیت را باز می‌کند و **is_open** را **True** می‌کند. متد **close** گیت را می‌بندد و **is_open** را **False** می‌کند. این دو متد توسط **Gate** فراخوانی می‌شوند و وضعیت فیزیکی گیت را کنترل می‌کنند.

۶. کلاس **Gate** منطق گیت

```
class Gate:
    def __init__(self):
        self.sensor = Sensor()
        self.motor = Motor()
```

این کلاس را به عنوان منطق اصلی گیت مترو طراحی کردم. **Gate** یک **Sensor** و یک **Motor** دارد و همه تصمیم‌گیری‌های ورود و خروج در این کلاس انجام می‌شود.

متد enter ورود مسافر

```
def enter(self, passenger):  
    self.sensor.detect()  
    if passenger.card.has_balance() == True:  
        passenger.card.deduct()  
        self.motor.open()  
        return True  
    else:  
        self.motor.close()  
        return False
```

این متد بخش منطق گیت است. ابتدا سنسور کارت را تشخیص می‌دهد، سپس موجودی بررسی می‌شود. اگر موجودی کافی باشد کرایه کسر شده و موتور گیت را باز می‌کند و True برمی‌گرداند. اگر موجودی کافی نباشد گیت بسته می‌ماند و False برمی‌گرداند.

متدهای exit_passenger / close خروج و بستن

```
def exit_passenger(self):  
    self.motor.open()  
def close(self):  
    self.motor.close()  
    self.sensor.reset()
```

متد exit_passenger برای خروج مسافر گیت را بدون بررسی موجودی باز می‌کند زیرا خروج رایگان است. متد close موتور را می‌بندد و سنسور را ریست می‌کند تا گیت آماده استفاده بعدی باشد.

۷. کلاس GUI رابط گرافیکی

```
class GUI:  
    def __init__(self, root):  
        self.root = root  
        self.root.title("Metro Gate Simulator")  
        self.gate = Gate()  
        self.card1 = Card("A", 10.0)  
        self.card2 = Card("B", 15.0)
```

```
self.card3 = Card("C", 20.0)
```

این کلاس را برای مدیریت کامل رابط گرافیکی طراحی کردم. یک **Gate** و سه کارت با موجودی‌های مختلف در سازنده ساخته می‌شوند. دیکشنری **cards** نام کارت‌ها را به اشیاء **Card** متصل می‌کند تا از **OptionMenu** راحت انتخاب شوند.

متد **build_ui** ساخت رابط کاربری

```
def build_ui(self):
    tk.Label(..., text="Metro Gate Simulator", font=("Arial",16,"bold"))
    self.canvas = tk.Canvas(self.root, width=200, height=120, bg="white")
    self.status_label = tk.Label(self.root, text="Gate: CLOSED", fg="red")
    self.name_entry = tk.Entry(self.root)
    card_menu = tk.OptionMenu(self.root, self.selected_card, ...)
    tk.Button(..., text="Enter", bg="lightblue", command=self.press_enter)
    tk.Button(..., text="Exit", bg="lightgreen", command=self.press_exit)
    self.log = tk.Text(self.root, height=7, width=38, state="disabled")
```

این متد تمام المان‌های بصری را می‌سازد. یک **Canvas** برای رسم گیت، یک **Label** برای نمایش وضعیت، یک **Entry** برای نام مسافر، یک **OptionMenu** برای انتخاب کارت، دو دکمه ورود و خروج و یک **Text** برای نمایش لاگ رویدادها.

متد **draw_gate** رسم تصویر گیت

```
def draw_gate(self, open_gate):
    self.canvas.delete("all")
    self.canvas.create_rectangle(30, 20, 50, 100, fill="gray")
    self.canvas.create_rectangle(150, 20, 170, 100, fill="gray")
    if open_gate == True:
        self.canvas.create_rectangle(38, 22, 44, 55, fill="green")
        self.canvas.create_text(100, 65, text="OPEN", fill="green")
    else:
        self.canvas.create_rectangle(48, 57, 152, 63, fill="red")
        self.canvas.create_text(100, 40, text="CLOSED", fill="red")
```

این متد تصویر گیت را روی Canvas رسم می‌کند. دو مستطیل خاکستری ستون‌های گیت هستند. اگر گیت باز باشد یک مستطیل سبز کوچک و متن OPEN نمایش داده می‌شود. اگر بسته باشد یک خط قرمز افقی بین ستون‌ها و متن CLOSED نشان داده می‌شود.

متد `press_enter` دکمه ورود

```
def press_enter(self):
    passenger = self.get_passenger()
    result = self.gate.enter(passenger)
    if result == True:
        msg = "[ENTER] " + passenger.name + " passed. Balance: $" + ...
        self.add_log(msg)
        self.update_balance()
        self.open_gate_then_close()
    else:
        messagebox.showwarning("Denied", "Not enough balance on card!")
```

این متد وقتی کاربر دکمه Enter می‌زند فراخوانی می‌شود. ابتدا یک مسافر از اطلاعات فرم ساخته می‌شود و به `gate.enter` داده می‌شود. اگر نتیجه `True` باشد پیام موفقیت به لاگ اضافه می‌شود، موجودی به‌روز می‌شود و گیت باز می‌شود. اگر `False` باشد پنجره هشدار نمایش داده می‌شود.

متد `open_gate_then_close` باز و بستن خودکار

```
def open_gate_then_close(self):
    self.draw_gate(True)
    self.status_label.config(text="Gate: OPEN", fg="green")
1    self.root.after(2000, self.close_gate)
    def close_gate(self):
        self.gate.close()
        self.draw_gate(False)
        self.status_label.config(text="Gate: CLOSED", fg="red")
```

متد `open_gate_then_close` گیت را باز رسم می‌کند و با `root.after(2000)` بعد از ۲ ثانیه `close_gate` را فراخوانی می‌کند. این شبیه‌سازی واقعی گیت است که چند ثانیه باز می‌ماند و بعد بسته می‌شود. گیت را می‌بندد و وضعیت Label را به قرمز برمی‌گرداند.

۸. نقطه ورودی برنامه

```
root = tk.Tk()
app = GUI(root)
root.mainloop()
```

این بلوک نقطه شروع برنامه است. یک پنجره `tkinter` ساخته می‌شود، آن را به `GUI` می‌دهد و با `mainloop()` برنامه را در حالت اجرا نگه می‌دارد تا کاربر پنجره را ببندد.